Alla Vostra Mobile App Work Log - September 8, 2026

Compiled summary for Tuesday, September 8, 2026 - focus: verified U.S. Eastern-time date handling, full pre-existing worklog review, free Apple account testing boundaries, Expo Go SDK 54 and iPhone limitations, Xcode and iOS platform setup, iPhone pairing and trust troubleshooting, Mac storage triage, Battle.net and Adobe temporary-file cleanup, CocoaPods installation, local Expo dev-client iOS build, Stripe native header workaround, physical-device installation, developer profile trust, Metro dev-client launch verification, clean testing-session shutdown, source commit capture, and current documentation artifact generation for Alla Vostra.

Generated in continuity with the established Alla Vostra worklog cadence: compact opening summary, source-material review, baseline, goals, grouped work-completed sections, validation, files touched, external account/device context, follow-up context, and output-artifact close.

This artifact is named alla_vostra_worklog_september8.pdf because the verified repository U.S. Eastern-time date used for generation was Tuesday, September 8, 2026, and no earlier September 8 root-level Alla Vostra worklog PDF existed before this documentation pass. The Eastern date was checked with TZ=America/New_York date and an external UTC-04:00 time check before generation. This PDF records the work window after alla_vostra_worklog_september7_part1.pdf and documentation commit ba481c9, through synced source commit 4d88913 Configure free iOS dev build testing, plus local native-build/device operations completed after that source commit.

Source Material Reviewed

- Reviewed all 61 pre-existing root-level Alla Vostra PDF worklogs from May, June, July, August, and September 7 Part 1 to preserve naming, page format, metadata, footer language, section order, validation style, files-touched inventory, external-device notes, follow-up cadence, and output-artifact close.
- Extracted recent worklogs with pdftotext and inspected metadata/page counts with pdfinfo, using the August 14, August 16, August 24, and September 7 Part 1 PDFs as the closest style anchors because they define the current long-form mobile-app testing and release documentation shape.
- Reviewed the current git history from a2ab388 Prepared native testing and optimized mobile assets through ba481c9 Generated the latest customary session pdf worklog for September 7 Part 1 and 4d88913 Configure free iOS dev build testing.
- Inspected the latest commit body, diff stat, and changed-file surface for 4d88913, confirming it included mob/eas.json development Apple Pay bypass env flags plus refreshed app and web project snapshots.
- Rechecked current repository status before this PDF generation pass and confirmed main was aligned with origin/main with no tracked working-tree changes at the start of documentation.
- Reviewed the mobile native-testing surface from the previous source pass: mob/app.json, mob/eas.json, mob/app.config.js, mob/metro.config.js, mob/package.json, mob/package-lock.json, mob/utils/stripePayments.js, and mob/utils/stripeNative.js.
- Reviewed the generated local iOS project state under mob/ios as an ignored development artifact created by npx expo run:ios, without treating generated native source as a tracked source change for this worklog.
- Inspected the local node_modules Stripe iOS interop header that was patched during troubleshooting so the worklog captures the temporary nature of the Xcode/Stripe enum workaround.

- Reviewed Xcode, iOS platform, CocoaPods, device-list, disk-space, Metro, and devicectl command outputs from the setup and validation flow so the worklog records concrete tool state rather than only conversational notes.
- Reviewed the user-supplied screenshots and messages covering Xcode Components, missing Developer Mode, pairing failure, system keychain access prompt, trust/profile setup, disk cleanup concern, CoreSimulator confusion, AdobeTemp concern, and final USB testing/eject flow.

Baseline Status Before September 8

- The September 7 Part 1 worklog left the project prepared for native testing and optimized mobile assets, but the physical iPhone Xcode run had not yet completed because Xcode was still downloading during that source window.
- The project already had Expo SDK 54, expo-dev-client, iOS bundle metadata, WebP runtime assets, a Stripe native/Expo Go shim, and an Apple-Pay-disabled configuration path, but the real local iPhone install still needed Xcode, CocoaPods, disk space, pairing, signing, and trust to line up.
- The user wanted to avoid paid Apple Developer Program enrollment for the moment, so the active goal was limited local native-device testing: viewing UI, exploring navigation, and confirming a development build can install and launch on a personal iPhone.
- Full TestFlight, App Store distribution, and production Apple Pay testing remained outside the free-account lane because those flows require Apple Developer Program credentials, app/capability setup, provisioning, and merchant entitlement handling.
- Expo Go on iPhone was not a complete fallback because iOS does not provide a direct old-version Expo Go SDK 54 download path like Android APK side-loading, and the app uses native Stripe modules that require a custom development or store build for complete checkout testing.
- Xcode installation alone was not enough initially: the iOS platform/simulator component had to be installed from Xcode Settings > Components before xcodebuild could use the iPhone and iOS SDK destination cleanly.
- Mac disk space was critically low during the first native build attempt, causing the pod/native setup path to fail until large non-project and cache folders were identified and cleaned.
- The iPhone pairing/trust path had multiple user-visible gates: Trust This Computer on iPhone, Developer Mode visibility/state, Xcode/device pairing, local signing keychain access, and the on-device developer profile trust step after installation.
- The generated iOS project and CocoaPods output were expected local build artifacts rather than durable app-source changes, while the EAS development env flags were the durable source-side support added for free-account local testing.
- The testing session started from a synced main branch but still needed a post-setup PDF worklog to record the device setup, cleanup, build failures, repairs, final successful launch, and caveats for future sessions.

Goals for September 8

- Confirm what iPhone testing can be done for free now and avoid pushing the user into paid Apple Developer enrollment before it is needed.
- Set up the Mac and iPhone for local native-device testing with Xcode, a free Apple account, USB connectivity, and Expo dev-client workflow.
- Clarify that Expo Go on iPhone cannot reliably be downgraded by direct SDK 54 download and cannot fully test native Stripe checkout for this app.
- Recover enough disk space for Xcode platforms, CocoaPods, iOS derived data, and the local native build without deleting unrelated software-development project folders.

- Explain large Mac storage candidates in plain terms, especially CoreSimulator and AdobeTemp, so the user can decide what is safe to remove.
- Remove Battle.net and AdobeTemp storage only after user permission and avoid deleting active project code or development source directories.
- Install and verify CocoaPods so npx expo run:ios can complete pod installation and native project build steps.
- Build, sign, install, trust, and launch Alla Vostra on Martin's iPhone as a local iOS development build.
- Start Metro in dev-client mode and launch the installed iPhone app against the local LAN bundle URL to prove the app can request and receive the JavaScript bundle.
- Stop the dev server cleanly after testing and leave a clear restart command plus follow-up caveats for persistent native iOS work.

Work Completed - Free Apple Account and Testing Scope

- Confirmed that a paid Apple Developer account is not required for the limited local testing lane used in this session, as long as the goal stays on personal-device installation and short-lived development signing.
- Separated free local testing from paid distribution: local Xcode/free-account testing can cover launch, UI review, navigation, layout, and some non-entitlement runtime behavior, while TestFlight, App Store submission, and production Apple Pay require paid Apple Developer Program enrollment.
- Recommended the free path for now because it lets the project make progress on native iPhone layout and navigation without spending money or creating App Store Connect distribution credentials prematurely.
- Clarified that Apple Pay should be disabled for this local free-account testing path because merchant identifiers, entitlements, provisioning, and real Apple Pay capability setup are not realistically covered by the free-account flow.
- Explained that Stripe card checkout and wallet payment validation should still be treated as native-development/store-build work, not Expo Go proof, because the app depends on @stripe/stripe-react-native native modules.
- Provided the practical testing target for the current phase: get Alla Vostra installed and launching on the physical iPhone, confirm the dev-client bundle loads, inspect screens/navigation, and postpone App Store-like payment verification until the proper paid account/capability lane exists.
- Kept external credentials and personal account secrets out of source and out of the worklog while documenting only the account categories, prompts, and blocker types.
- Updated the durable development build configuration in mob/eas.json through commit 4d88913 so the EAS development profile now carries ALLA_VOSTRA_DISABLE_APPLE_PAY_ENTITLEMENT=1 and EXPO_PUBLIC_DISABLE_APPLE_PAY=1.
- Committed the snapshot refresh and development env flag work as 4d88913 Configure free iOS dev build testing, leaving the repository clean and synced before this PDF generation pass.

Work Completed - Xcode, iOS Platform, and iPhone Pairing

- Guided the Xcode installation path after the user installed Xcode, including the need to use the full Xcode toolchain rather than only Command Line Tools for physical iPhone native testing.
- Identified that the initial Xcode setup was incomplete because the iOS platform/simulator component was missing, then directed the user to Xcode Settings > Components to install the iOS 26.5 platform package.
- Confirmed from the user screenshot and xcodebuild output that the iOS 26.5.1/iOS 26.5

Simulator component was installed and that the active Xcode toolchain later reported Xcode 26.6 build 17F113 with iOS 26.5 SDKs available.
- Worked through the iPhone trust and pairing failure messages, including the system prompt stating that the Mac could not pair with the device and needed the USB cable disconnected/reconnected with instructions followed on the phone.
- Explained the Developer Mode issue when the user could not find Developer Mode under Privacy & Security, noting that Developer Mode only appears after the device is connected, paired, and has had a developer app or Xcode interaction that triggers the setting.
- Confirmed later device details from devicectl during the successful flow: Martin's iPhone was paired, wired, developer mode was enabled, and the physical UDID was available for the local build destination.
- Handled the macOS keychain prompt from codesign asking to access the Apple Development key for Martin Prahl, explaining that it is expected during local signing and that Allow or Always Allow lets the build sign the app.
- Handled the post-install invalid-signature/trust failure by directing the user to iPhone Settings > General > VPN & Device Management to trust the Apple Development profile for the local developer account.
- Relaunched the app after the trust step and confirmed the installed application could start on the iPhone rather than remaining blocked by profile trust or code-signing state.

Work Completed - Storage Triage and Cleanup

- Scanned the Mac for large storage candidates when Xcode/native build setup ran into low disk space, keeping the recommendation focused on large folders that were either caches, app support data, or removable tooling artifacts rather than active Alla Vostra source.
- Explained CoreSimulator as Apple's local storage for iOS simulator runtimes and simulator devices: it is installed by Xcode for running simulated iPhones/iPads on the Mac and can legitimately occupy many gigabytes.
- Clarified that the visible Xcode Components size can differ from du output because CoreSimulator includes APFS volumes, mounted runtime images, devices, caches, and filesystem accounting that do not always match the single component number shown in Xcode.
- Identified a large old iOS 26.3.1 simulator runtime around 8.39 GB in Xcode UI and a larger CoreSimulator footprint on disk, while advising that deleting old simulator runtimes is safer than deleting project source when more space is needed.
- Explained AdobeTemp as temporary Adobe installer/updater/cache state under /System/Volumes/Data/.adobeTemp, generally removable when no Adobe installer/update is actively running, but still requiring admin permission because of its system-volume location.
- Removed Battle.net storage after user approval, including Battle.net data under the user Library Application Support area and /Users/Shared/Battle.net through an admin-authorized removal path.
- Removed /System/Volumes/Data/.adobeTemp after user permission and admin authorization, then verified the targeted AdobeTemp and Battle.net locations were gone.
- Cleared safe cache/storage areas during the setup flow, including npm cache, Gradle caches, VS Code ShipIt/update cache, Google cache, and other disposable local caches where permissions allowed.
- Improved available disk from the critically low native-build state to roughly 14-16 GiB available after cleanup and build operations, enough to complete CocoaPods, build, install, and run the local iOS app during this session.

- Left known large dev-related storage candidates documented for future intentional cleanup only: Android AVDs, Android SDK, Xcode/CoreSimulator runtimes, Adobe application support, and VS Code web storage.

Work Completed - Native iOS Build and Stripe Workaround

- Installed CocoaPods through Homebrew and verified pod --version reported 1.17.0, unblocking the iOS pod install phase required by the generated Expo native project.
- Started the local native build from mob with npx expo run:ios --device "Martin’s iPhone" and the Apple-Pay-disabled development environment variables active.
- Allowed Expo prebuild/run tooling to create the ignored mob/ios native project locally, keeping it as generated development output rather than a tracked source commit.
- Recovered from the first build failure caused by lack of disk space during pod/native setup by clearing storage and rerunning the iOS build path after enough room was available.
- Recovered from the Xcode platform failure by installing the iOS 26.5 SDK/simulator component in Xcode and rerunning the build once xcodebuild could see the required destination/platform.
- Diagnosed the Stripe iOS build failure where STPPaymentStatus was redeclared with a different underlying enum type, indicating a local Xcode/Stripe interop mismatch in the installed package headers.
- Patched mob/node_modules/@stripe/stripe-react-native/ios/StripeSwiftInterop.h locally from typedef NS_ENUM(NSUInteger, STPPaymentStatus) to typedef NS_ENUM(NSInteger, STPPaymentStatus), matching the Stripe SDK declaration expected by the current build.
- Reran the native iOS build after the Stripe header patch and reached a successful build/install on Martin’s iPhone for Alla Vostra version 1.0.3 / com.allavostra.app.
- Kept the Stripe header change documented as a temporary ignored node_modules workaround rather than a durable source fix; if node_modules is reinstalled, the same issue may need a persistent patch-package style solution or dependency update.

Work Completed - Installed App Launch and Dev Client Session

- Confirmed the app installed on the iPhone as Alla Vostra with bundle identifier com.allavostra.app and version 1.0.3.
- Resolved the first launch failure caused by untrusted local development signing by having the user trust the developer profile on the iPhone, then relaunched the application with devicectl.
- Started Metro in the mobile project with npx expo start --dev-client --host lan so the installed development client could load the Expo Router entry bundle from the Mac.
- Checked the generated iOS Info.plist URL schemes and confirmed the app included allavostra, com.allavostra.app, and exp+alla-vostra, allowing a dev-client payload URL to target the installed app.
- Used the Mac LAN address 192.168.12.33 and launched the installed iPhone app with the payload URL exp+alla-vostra://expo-development-client/?url=http%3A%2F%2F192.168.12.33%3A8081.
- Watched Metro bundle node_modules/expo-router/entry.js for iOS and confirmed it completed successfully in 30143 ms with 1642 modules bundled.
- Confirmed through devicectl that Alla Vostra was installed and that the AllaVostra process was running on the physical iPhone after launch.
- Stopped Metro cleanly when the user asked to eject, then confirmed the server session exited and no process was listening on TCP port 8081 during the final documentation checks.
- Clarified that the iPhone does not need a disk-style eject step after this USB

development session; after Metro stopped, the phone could be unplugged normally.

Validation and Checks Performed

- Verified the U.S. Eastern-time date before naming this artifact: TZ=America/New_York date returned Tuesday, September 8, 2026 EDT (-0400), and an external UTC-04:00 time check was also used for the same documentation decision.
- Ran git status --short --branch before this documentation pass and confirmed main was clean and aligned with origin/main.
- Reviewed git log and git show for 4d88913, confirming the latest synced source commit is Configure free iOS dev build testing with three tracked files changed: alla_vostra_APP_PROJECT_SNAPSHOT.txt, alla_vostra_WEB_PROJECT_SNAPSHOT.txt, and mob/eas.json.
- Ran xcodebuild -version and confirmed Xcode 26.6 build 17F113 is the active toolchain.
- Ran xcodebuild -showsdks and confirmed iOS 26.5 and iOS Simulator 26.5 SDKs are installed, along with the current macOS, tvOS, visionOS, watchOS, and DriverKit SDK listings.
- Ran pod --version and confirmed CocoaPods 1.17.0 is installed.
- Checked disk space after cleanup/build work and saw roughly 14 GiB available on /System/Volumes/Data and /, enough for the completed native test but still tight for repeated Xcode builds.
- Checked devicectl list devices after the user unplugged/ejected; Martin’s iPhone was listed as unavailable, which is expected after physical disconnection.
- Checked that Metro was no longer listening on TCP port 8081 after the dev server was stopped.
- Confirmed the local generated iOS project exists under mob/ios at about 937 MB, and confirmed the local StripeSwiftInterop.h workaround still uses NSInteger for STPPaymentStatus.
- Earlier in the setup flow, confirmed the iPhone was wired, paired, developer mode enabled, and available as the Xcode destination before build/install/launch.
- Earlier in the setup flow, confirmed the app installed and launched on the physical iPhone, with Metro successfully bundling the iOS Expo Router entry point.
- Did not run a new build during this final documentation pass because the successful build/install/launch had already completed and the user had stopped the testing session.
- Did not run a real Stripe card charge, Apple Pay transaction, Google Pay transaction, PayPal capture, TestFlight install, App Store install, or production iOS payment validation during this free local-device testing window.

Files Touched During the September 8 Window

- mob/eas.json - durable source change in commit 4d88913 adding development env flags for ALLA_VOSTRA_DISABLE_APPLE_PAY_ENTITLEMENT=1 and EXPO_PUBLIC_DISABLE_APPLE_PAY=1.
- alla_vostra_APP_PROJECT_SNAPSHOT.txt - refreshed in commit 4d88913 after the native testing setup pass, capturing current mobile configuration, WebP assets, dev-client support, and Stripe shim references.
- alla_vostra_WEB_PROJECT_SNAPSHOT.txt - refreshed in commit 4d88913, including the September 7 Part 1 worklog PDF in the root project file tree.
- mob/ios/ - generated locally by npx expo run:ios during the device build workflow; this is ignored/generated native output and was not committed as durable source.
- mob/node_modules/@stripe/stripe-react-native/ios/StripeSwiftInterop.h - patched locally inside ignored node_modules to resolve the STPPaymentStatus enum type mismatch for the current Xcode/Stripe build.
- Homebrew/CocoaPods local environment - changed outside the repository when CocoaPods was

installed to support iOS pod installation.
- ~/Library/Application Support/Battle.net and /Users/Shared/Battle.net - removed outside the repository after user approval during disk cleanup.
- /System/Volumes/Data/.adobeTemp - removed outside the repository after user approval/admin authorization during disk cleanup.
- Local cache directories including npm, Gradle, VS Code ShipIt/update cache, and Google cache - cleaned where safe and permission-allowed to restore enough native-build disk headroom.
- alla_vostra_worklog_september8.pdf - generated by this documentation pass as the latest root-level PDF worklog artifact for the project.

External Configuration and Device State

- The active iPhone testing device was Martin’s iPhone, previously detected as an iPhone 14-class device over wired USB and used as the physical Xcode destination for the local build.
- The app installed under bundle identifier com.allavostra.app with visible app name Alla Vostra and version 1.0.3.
- The local Apple Development signing identity for Martin Prahl was used by codesign, and macOS requested login keychain permission before signing could proceed.
- The iPhone required manual trust of the local developer profile before the installed app could launch, which is expected for free/local development signing.
- No paid Apple Developer Program enrollment was completed during this work window, and no App Store Connect, TestFlight, Apple merchant, or Apple Pay production configuration was created.
- Apple Pay was intentionally disabled for the local development build path through source/env controls so local UI and navigation testing could proceed without merchant entitlement blockers.
- The dev-client LAN URL used during successful testing pointed at the Mac on 192.168.12.33:8081; future runs may need the current Mac LAN IP if the network address changes.
- The iPhone is currently disconnected/unavailable according to devicectl after the user asked to eject and the device was unplugged or otherwise no longer available to the Mac.
- Metro is currently stopped, so a future iPhone dev-client session should restart it from mob with npx expo start --dev-client --host lan before launching or reloading the installed app.
- No Stripe, PayPal, Vercel, Postmark, Google Play, App Store, or Apple account secret values were added to source or recorded in this PDF.

Follow-up Context for Future Sessions

- To repeat the local iPhone dev-client session, reconnect the iPhone, unlock it, trust the Mac if prompted, start Metro from mob with npx expo start --dev-client --host lan, and launch the installed Alla Vostra dev build.
- If the app cannot reach Metro on a future run, confirm the iPhone and Mac are on the same Wi-Fi/network, check the Mac LAN IP, verify port 8081 is listening, and relaunch with the current exp+alla-vostra dev-client URL.
- If node_modules is reinstalled and the Stripe enum build error returns, convert the local StripeSwiftInterop.h change into a durable patch or update the Stripe React Native/iOS dependency path instead of relying on a forgotten manual edit.
- Keep the Apple Pay disabled flags active for free/local iPhone testing until a paid Apple Developer account, merchant identifier, capability, provisioning profile, and native Apple Pay test plan are ready.

- When paid Apple Developer enrollment is available, create the proper App Store/TestFlight path and retest the installed iOS app with Apple Pay capability, Stripe card entry, PayPal approval/capture, order confirmation, and email delivery.
- Keep at least 20-30 GiB free before repeated Xcode builds if possible; iOS platforms, DerivedData, simulators, pods, and archives can consume space quickly.
- If more space is needed, review old Xcode simulator runtimes and unused Android AVDs intentionally; do not delete active project directories or source repositories just because they are large.
- The old iOS 26.3.1 simulator runtime remains a reasonable candidate for deletion if it is not needed, but the current iOS 26.5 platform should stay installed for the present Xcode/device toolchain.
- Treat Expo Go on Android as useful for visual/layout passes, but treat native iPhone checkout and payment surfaces as development-build/TestFlight/store-build responsibilities.
- Commit this September 8 PDF as a documentation-only artifact if following the recent project cadence of source/config commit first, PDF worklog commit second.

Output Artifact

- Created alla_vostra_worklog_september8.pdf at the repository root as the latest customary PDF worklog for the Alla Vostra project.
- Used the established root-level worklog filename pattern because this was the first September 8 worklog artifact, so no part suffix was required.
- Preserved the current letter-size PDF convention, OpenAI Codex metadata, Codex standard-library PDF generator creator/producer values, footer format, concise opening summary, section ordering, bullet-heavy documentation style, validation language, files-touched detail, external account/device context, follow-up section, and output-artifact close.
- This PDF records the September 8 free iOS native-device setup work: Apple account/testing-scope decisions, Xcode and iOS platform installation, device pairing and trust repair, storage cleanup, CocoaPods setup, Stripe header workaround, successful physical-iPhone build/install/launch, Metro dev-client bundle verification, clean eject/shutdown, synced source state through 4d88913, and remaining native iOS testing risks.